Python Programming Mid-Term Project Report

Project Title

Snake Game using Pygame

Track Name

Track 1 — Game (Pygame)

Student Information

Name: Tasnim Sultana
ID: 22-47560-2
Name: Thiyab Rahman
ID: 23-50838-1

Section: B

1. Introduction

This project is a simple Snake Game developed using Python and the Pygame library. The game allows the player to control a snake that moves across the screen and collects apples. Each time the snake eats an apple, it grows in size. The project demonstrates basic programming concepts and Object-Oriented Programming (OOP).

2. Objective

The objectives of this project are:

- To develop a playable game using Python
- To apply Object-Oriented Programming concepts
- To use an external library (pygame)
- To organize code in a modular way

3. Development Environment

A virtual environment was used for this project.

Setup Instructions

1. Create virtual environment:
   python -m venv .venv
2. Activate environment:
   .venv\Scripts\activate
3. Install required library:
   pip install pygame
4. Run the project:
   python main.py
5. Generate requirements file:
   pip freeze > requirements.txt

4. External Library Used

- pygame

5. Project Structure

The project is divided into multiple files:

- main.py → main game loop and display
- snake.py → snake movement and body handling
- apple.py → apple generation and position
- settings.py → constants such as screen size, colors
- requirements.txt → list of dependencies

6. Object-Oriented Design

The project follows Object-Oriented Programming principles.

Classes Used:

- Main
- Snake
- Apple
- Settings module

OOP Features:

- Use of classes and objects
- Use of constructors (**init**)
- Methods for handling behaviors
- Composition (Main class uses Snake and Apple objects)

7. Features Implemented

- Snake movement using keyboard input
- Continuous movement using timer
- Apple appears in random positions
- Snake grows after eating apple
- Game window rendering using pygame
- Background grid system
- Modular file structure

8. Game Flow

1. Game starts and window opens
2. Snake appears at starting position
3. Player controls snake direction
4. Snake moves continuously
5. Snake eats apple and grows
6. Apple respawns in new position
7. Game continues until exit

9. Validation

- Apple does not spawn on snake body
- Game handles exit properly
- Code is structured to avoid crashes

10. Code Quality

- Clean and readable code
- Proper naming conventions
- Separate files for different logic
- Reusable settings

11. Limitations

- No score system
- No game over condition
- No restart option
- No data saving (JSON/CSV)

12. Future Improvements

- Add score system
- Add game over screen
- Add restart functionality
- Save high score using JSON
- Add difficulty levels

13. Conclusion

This project demonstrates the use of Python and Object-Oriented Programming to develop a simple game. It also shows how external libraries like pygame can be used to create interactive applications.

14. Screenshots

```python
class Main:
    def __init__(self):

        # general
        pygame.init()
        self.display_surface = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
        pygame.display.set_caption('Snake Game')

        # game objects
        self.bg_rects = [pygame.Rect((COL + int(row % 2 == 0)) * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE)
                         for COL in range(0, COLS, 2) for row in range(ROWS)]
        self.snake = Snake()
        self.apple = Apple(self.snake)

        # timer
        self.update_event = pygame.event.custom_type()
        pygame.time.set_timer(self.update_event, 200)
        self.game_active = False
```

```python

    def run(self):
        while True:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    exit()

                if event.type == self.update_event and self.game_active:
                    self.snake.update()

                if event.type == pygame.KEYDOWN and not self.game_active:
                    self.game_active = True

            # updates
            if self.game_active:
                self.input()
                self.collision()

            # drawing
            self.display_surface.fill('black')
            # self.draw_bg()
            self.snake.draw()
            self.apple.draw()
            pygame.display.update()


if __name__ == '__main__':
    main = Main()
```

Snake Game

Snake Game

Snake Game